

参考資料

Base Rを用いたグラフ

医療データ科学実習

(Practice of Biomedical Data Science)

第7回 2026年6月2日

医療データ科学

matsui.kota.8k@kyoto-u.ac.jp

臨床統計学

emoto.ryo.5m@kyoto-u.ac.jp

Rの組み込み関数を用いた グラフィックス

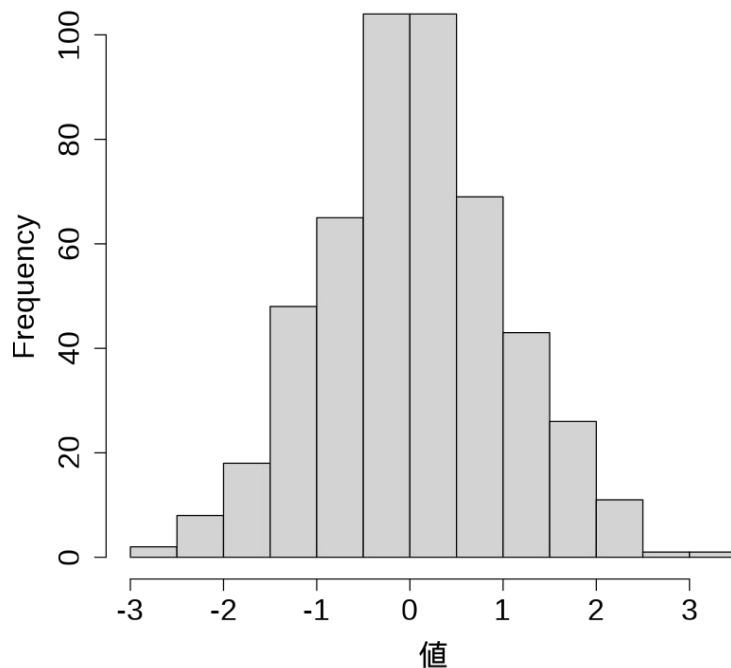
- 1次元の連続変数の経験分布
(ヒストグラム, 経験累積分布)
- 箱ひげ図
- 複数のプロットの重ね描き

hist によるデータのヒストグラム作成

ヒストグラム (histogram)

- 連続的な数値データの分布(頻度)を視覚的に表現する棒グラフの一種
- データがどの範囲に多く存在するか(分布の形)を把握するためによく使われる
 - 棒の幅=ビンの幅(通常は等間隔)
 - 棒の高さ=ビンに含まれるデータの数

dataのヒストグラム

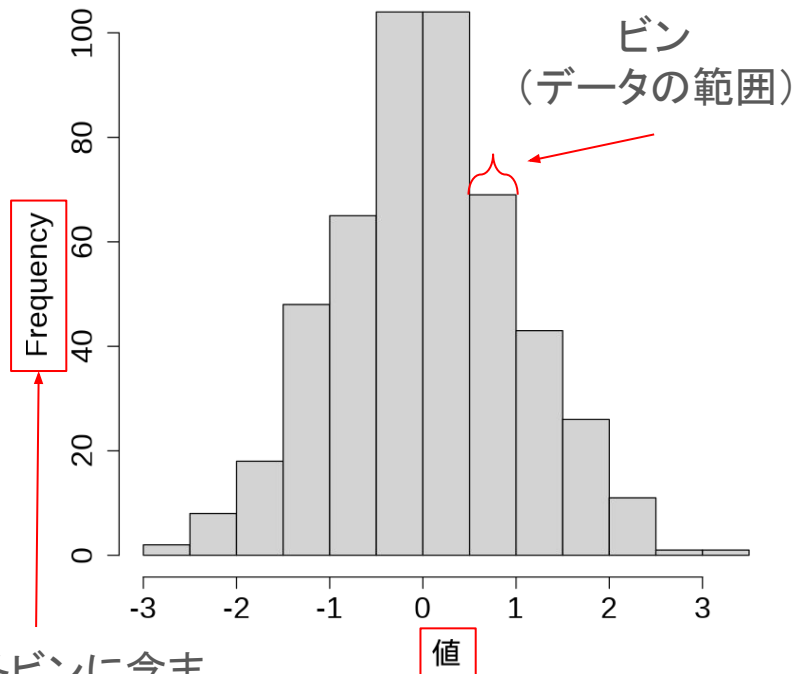


hist によるデータのヒストグラム作成

ヒストグラム (histogram)

- 連続的な数値データの分布(頻度)を視覚的に表現する棒グラフの一種
- データがどの範囲に多く存在するか(分布の形)を把握するためによく使われる
 - 棒の幅=ビンの幅(通常は等間隔)
 - 棒の高さ=ビンに含まれるデータの数

dataのヒストグラム



縦軸は各ビンに含まれるデータの件数

横軸はデータの値

hist によるデータのヒストグラム作成

ヒストグラム描画のサンプルコード

```
set.seed(123)
data <- rnorm(500)
par(cex.main = 3, cex.lab = 2, cex.axis = 2)
hist(data, breaks=10, main='dataのヒストグラム', xlab='値')
```

hist によるデータのヒストグラム作成

ヒストグラム描画のサンプルコード

```
set.seed(123)
```

 → 乱数シードを固定

```
data <- rnorm(500)
```

 → 標準正規分布からの乱数を500個生成

```
par(cex.main = 3, cex.lab = 2, cex.axis = 2)
```

```
hist(data, breaks=10, main='dataのヒストグラム', xlab='値')
```

→ タイトル, 縦軸, 横軸の文字サイズ

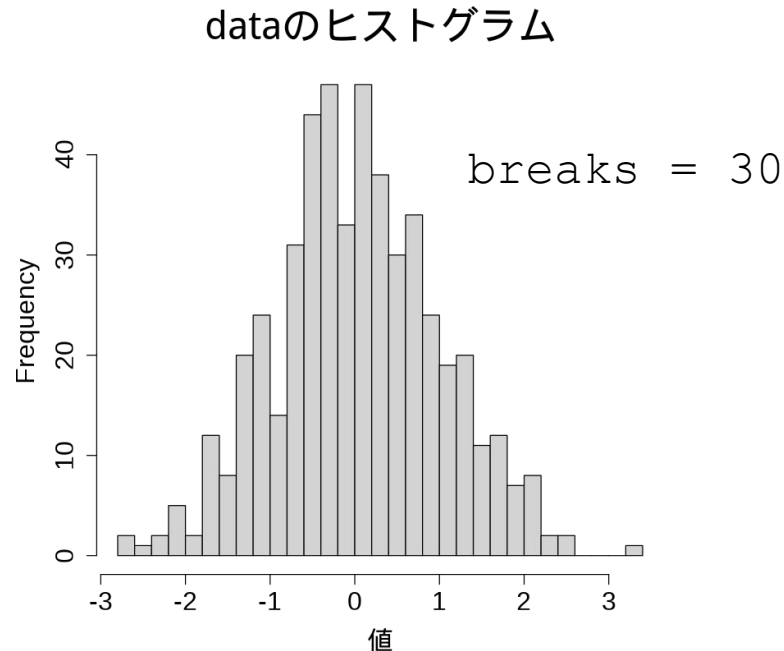
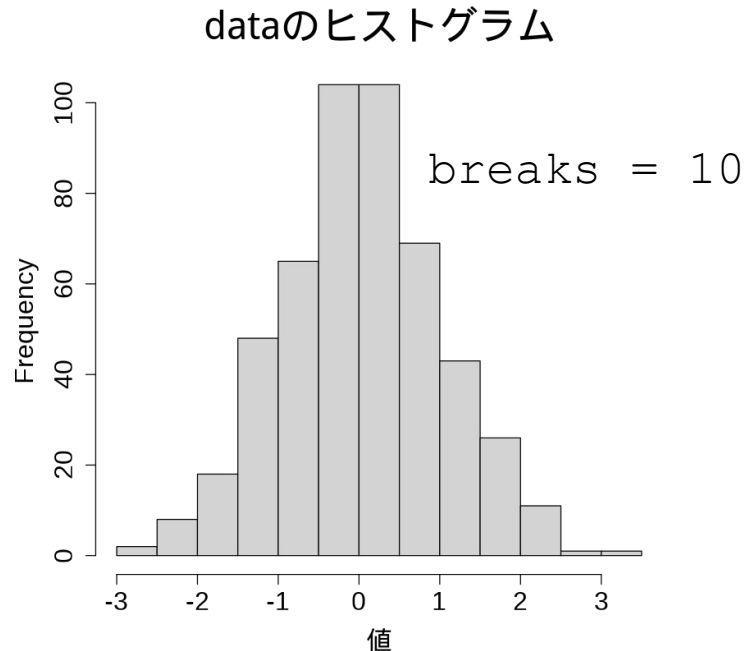
→ ヒストグラムを描画するRのベース関数

- data: ヒストグラムを作りたいデータ集合
- breaks: ビンの数
- main: プロットのタイトル
- xlab: 横軸のラベル

演習: RStudioで上記のコードを実行してヒストグラムを描画してみよう

hist によるデータのヒストグラム作成

ビンの違いによるヒストグラムの違い



binを細かくするほど局所的な形状が正確に表現されるようになる

演習 : ビンの数を増やすとヒストグラムの形状がどう変化していくか見てみよう

hist によるデータのヒストグラム作成

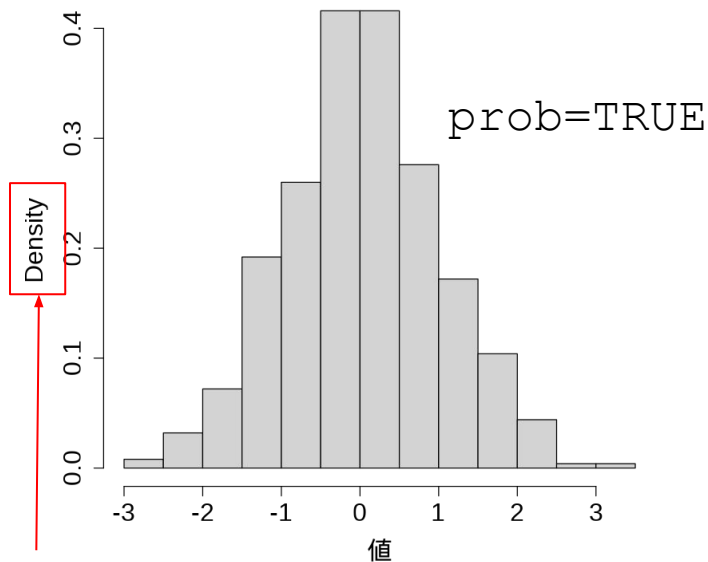
prob オプションによる確率密度の描画

```
hist(data, breaks=10, prob=TRUE, main='dataのヒストグラム', xlab='値')
```

dataのヒストグラム

prob オプションで縦軸を頻度↔確率密度と切り替えられる

- prob=FALSE: 縦軸が頻度に(デフォルト)
- prob=TRUE: 縦軸が確率密度に



縦軸が Density に変わる

演習: prob オプションを切り替えてヒストグラムを描画してみよう

ecdf によるデータの経験累積分布関数の可視化

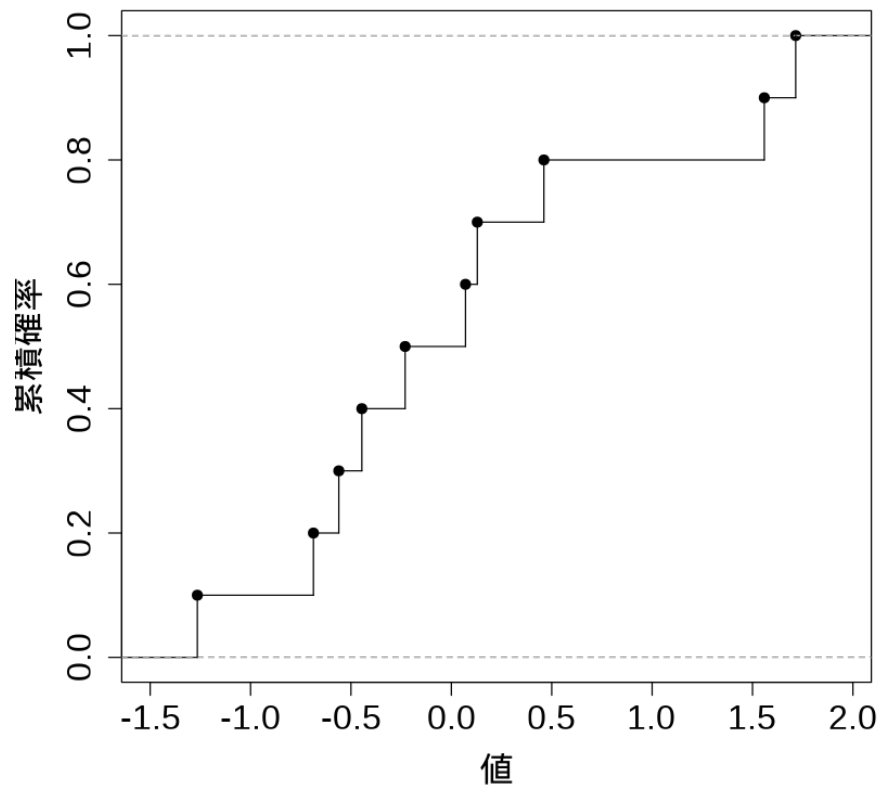
累積分布関数(cumulative distribution function, cdf)

確率変数 X の累積分布関数 $F(X)$ は、ある実現値 x に対して

$$F(x) = \Pr(X \leq x)$$

で定義される

データのECDF



ecdf によるデータの経験累積分布関数の可視化

累積分布関数(cumulative distribution function, cdf)

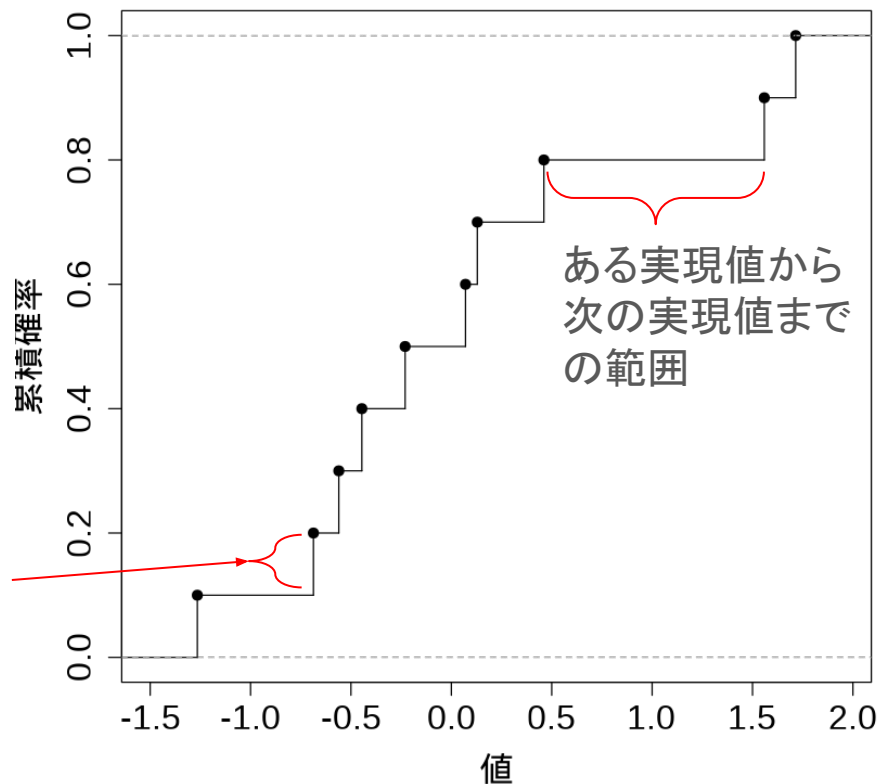
確率変数 X の累積分布関数 $F(X)$ は、ある実現値 x に対して

$$F(x) = \Pr(X \leq x)$$

で定義される

観測値が現れたとき、その点より小さいか等しい値のデータの割合(累積確率)が増える → 結果として、cdfの値(縦軸)が「ジャンプ」する

データのECDF



ecdf によるデータの経験累積分布関数の可視化

cdf描画のサンプルコード

```
data <- rnorm(10)
par(cex.main = 3, cex.lab = 2, cex.axis = 2)
plot(ecdf(data), main='データのECDF', xlab='値', ylab='累積確率')
```

ecdf によるデータの経験累積分布関数の可視化

cdf描画のサンプルコード

```
data <- rnorm(10)
```

→ 標準正規分布から乱数を10個生成

```
par(cex.main = 3, cex.lab = 2, cex.axis = 2)
```

```
plot(ecdf(data), main='データのECDF', xlab='値', ylab='累積確率')
```

→ data の経験累積分布関数 (ecdf) を計算

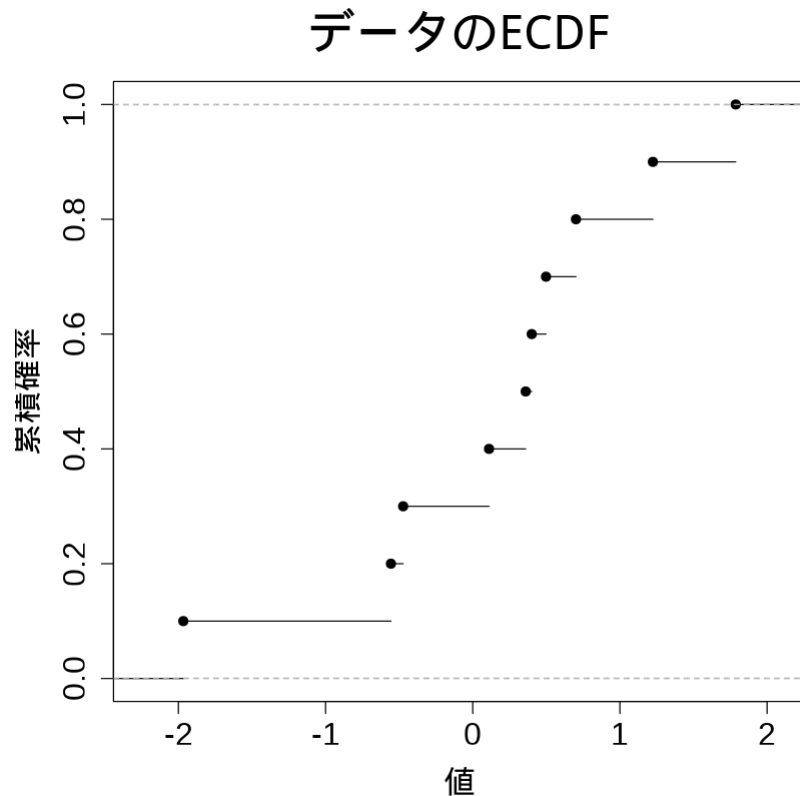
→ 計算された ecdf をプロット

ecdf によるデータの経験累積分布関数の可視化

cdf描画のプロット結果

演習 : RStudioでコードを実行して標準正規分布の ecdf をプロットしてみよう

*デフォルトではジャンプを表す縦の線は入らないことに注意



ecdf によるデータの経験累積分布関数の可視化

ecdfの縦線を自動で入れる方法

```
plot(ecdf(data), main = "データのECDF", xlab = "値", ylab = "累積確率",  
verticals = TRUE,  
do.points = TRUE)
```

*コードの改行に注意(提示しているサンプルコードはスライドに収めるために改行している)

ecdf によるデータの経験累積分布関数の可視化

ecdfの縦線を自動で入れる方法

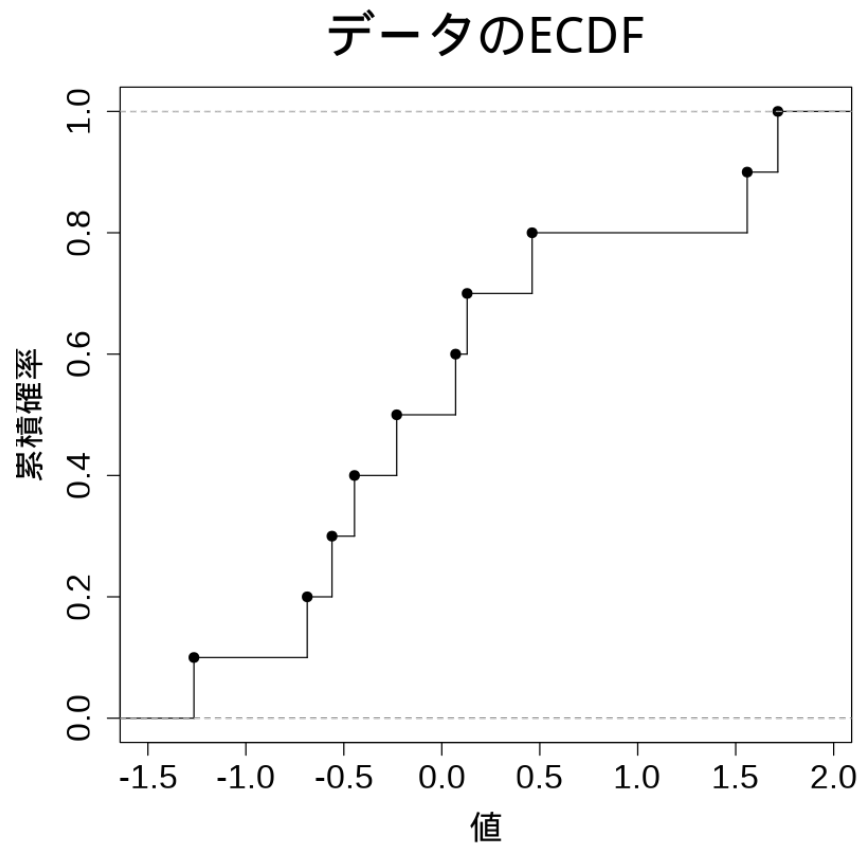
```
plot(ecdf(data), main = "データのECDF", xlab = "値", ylab = "累積確率",  
verticals = TRUE, do.points = TRUE)
```

→ plot 関数の verticals オプションを TRUE にする

→ do.points オプションでデータ点の表示も指定できる

ecdf によるデータの経験累積分布関数の可視化

実行結果

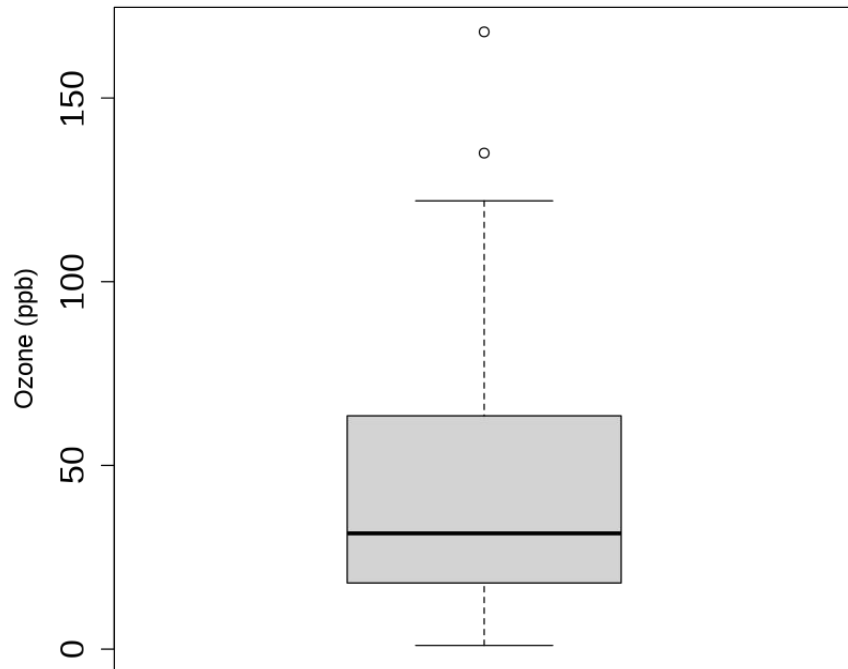


boxplot によるデータの箱ひげ図の描画

箱ひげ図 (box plot)

- データの分布・ばらつき・外れ値を視覚的に要約するためのグラフ
- **5つの要約統計量**をもとに構成され、データの中心・広がり・非対称性・外れ値などを直感的に把握できる

Ozone濃度のボックスプロット

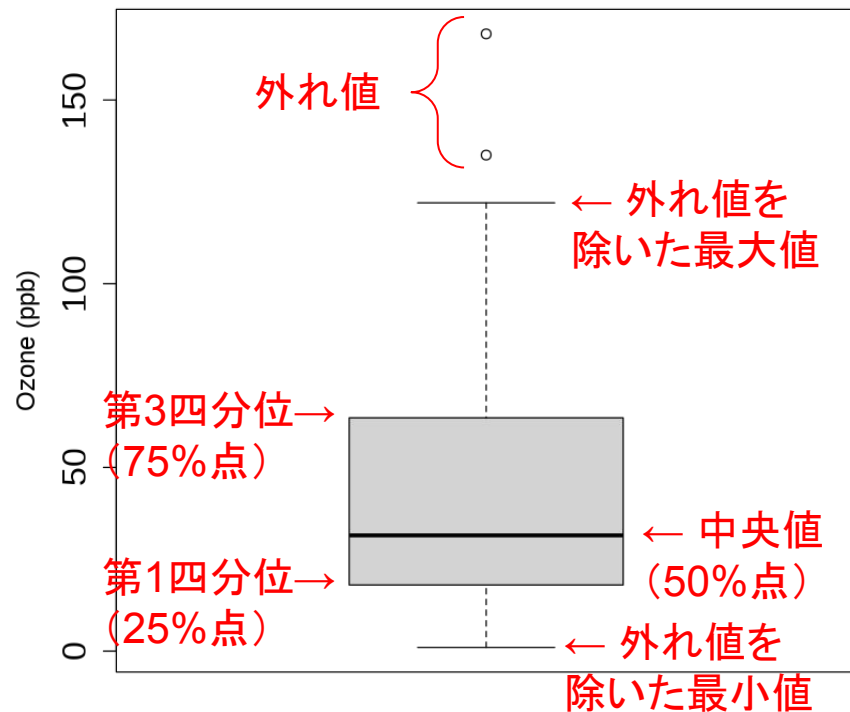


boxplot によるデータの箱ひげ図の描画

箱ひげ図 (box plot)

- データの分布・ばらつき・外れ値を視覚的に要約するためのグラフ
- 5つの要約統計量をもとに構成され、データの中心・広がり・非対称性・外れ値などを直感的に把握できる

Ozone濃度のボックスプロット



boxplot によるデータの箱ひげ図の描画

箱ひげ図描画のサンプルコード

```
# Ozone列の欠損値を除去
```

```
ozone <- na.omit(airquality$Ozone)
```

```
# ボックスプロットを描画
```

```
par(cex.main = 2, cex.lab = 1.5, cex.axis = 2)
```

```
boxplot(ozone, main = "Ozone濃度のボックスプロット", ylab = "Ozone (ppb)")
```

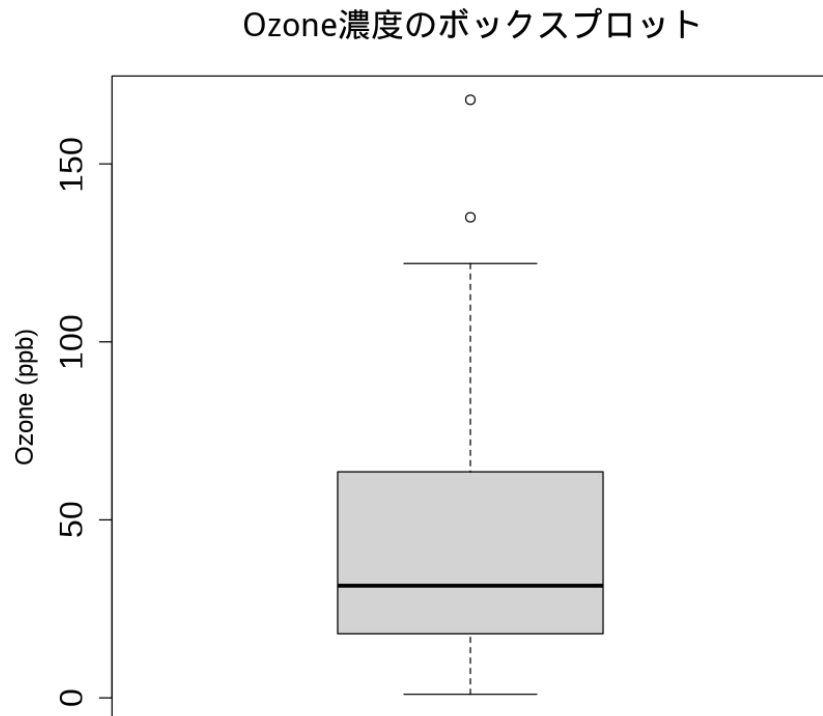
実は、boxplot には na.action オプションがあり、デフォルトでは欠測値は自動的に無視される（つまり最初の欠損値除去はなくても良い）

boxplot によるデータの箱ひげ図の描画

箱ひげ図描画の実行結果

演習 : RStudioでコードを実行して箱ひげ図をプロットしてみよう

演習 : 欠損値の処理を入れた場合と入れない場合でプロットが変わるかどうかを確認しよう



複数のプロットを重ねて表示する

- `plot` 関数を用いる場合の重ね描き
 - `par(new = TRUE)` で次の描画を前の上に重ねる
 - `lines()` や `points()` を使って追加描画
 - `matplot()` を使って複数系列を一括で描画
- `plot` 以外の関数(e.g. `hist`, `boxplot`, ...)
 - 各関数の `add = TRUE` オプションを用いる

`par(new = TRUE)` で次の描画を前の上に重ねる

プロットの重ね合わせのサンプルコード

```
# 基本のプロット（例：sinカーブ）
```

```
x <- seq(0, 2 * pi, length.out = 100)
```

```
par(cex.main = 2, cex.lab = 1.5, cex.axis = 2)
```

```
plot(x, sin(x), type = "l", col = "blue", lwd = 2,  
      ylim = c(-1.5, 1.5), main = "sin と cos の重ね描き")
```

`par(new = TRUE)` で次の描画を前の上に重ねる

プロットの重ね合わせのサンプルコード

基本のプロット（例：sinカーブ）

```
x <- seq(0, 2 * pi, length.out = 100)
```

 → 横軸の生成： $[0, 2\pi]$

```
par(cex.main = 2, cex.lab = 1.5, cex.axis = 2)
```

```
plot(x, sin(x), type = "l", col = "blue", lwd = 2,  
      ylim = c(-1.5, 1.5), main = "sin と cos の重ね描き")
```

横軸データと
縦軸データ

プロット線のスタイル
("l"は線だけ)

線の色

線の太さ

`par(new = TRUE)` で次の描画を前の上に重ねる

プロットの重ね合わせのサンプルコード

```
# 基本のプロット（例：sinカーブ）
```

```
x <- seq(0, 2 * pi, length.out = 100)
```

```
par(cex.main = 2, cex.lab = 1.5, cex.axis = 2)
```

```
plot(x, sin(x), type = "l", col = "blue", lwd = 2,  
     ylim = c(-1.5, 1.5), main = "sin と cos の重ね描き")
```

→ 縦軸のプロット範囲(-1.5~1.5の範囲でプロットを作成)

`par(new = TRUE)` で次の描画を前の上に重ねる

プロットの重ね合わせのサンプルコード

```
# 新しいプロットを重ねる  
par(new = TRUE)
```

直前のプロットコードと直後のプロットコードの間に
上をはさむ

`par(new = TRUE)` で次の描画を前の上に重ねる

プロットの重ね合わせのサンプルコード

重ねるプロット（例：cosカーブ）

```
plot(x, cos(x), type = "l", col = "red", lwd = 2,  
     axes = FALSE, xlab = "", ylab = "", ylim = c(-1.5, 1.5))
```

→ 二つ目以降のプロットでは、軸ラベルなどはつけない
(つけると文字が重なって表示されてしまう)

`par(new = TRUE)` で次の描画を前の上に重ねる

プロットの重ね合わせのサンプルコード(全体)

```
# 基本のプロット (例: sinカーブ)
```

```
x <- seq(0, 2 * pi, length.out = 100)
```

```
par(cex.main = 2, cex.lab = 1.5, cex.axis = 2)
```

```
plot(x, sin(x), type = "l", col = "blue", lwd = 2,  
      ylim = c(-1.5, 1.5), main = "sin と cos の重ね描き")
```

```
# 新しいプロットを重ねる
```

```
par(new = TRUE)
```

```
# 重ねるプロット (例: cosカーブ)
```

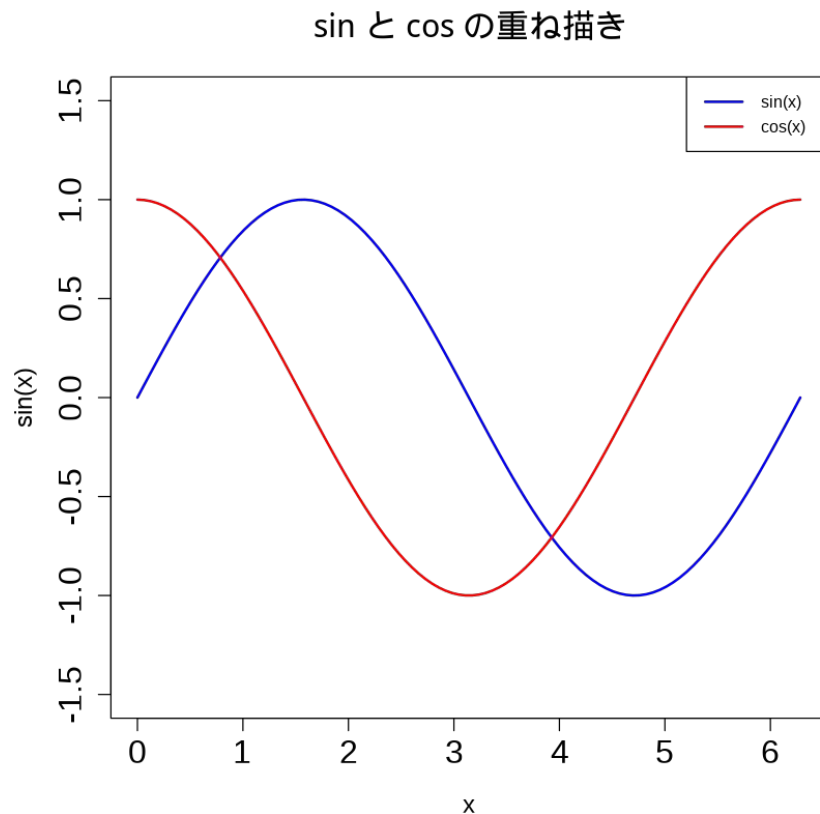
```
plot(x, cos(x), type = "l", col = "red", lwd = 2,  
      axes = FALSE, xlab = "", ylab = "", ylim = c(-1.5, 1.5))
```

`par(new = TRUE)` で次の描画を前の上に重ねる

プロットの重ね合わせの実行結果

演習 : RStudioでコードを実行してプロットを重ねて表示してみよう

*サインカーブのプロットとコサインカーブのプロットを順番に実行すると「プロットが重なる」感じが分かる



hist 関数で add = TRUE オプションを用いる

サンプルコード

```
# サンプルコード1: 2つの正規分布の密度を重ね描き
set.seed(1)
x1 <- rnorm(30, mean=0)
x2 <- rnorm(30, mean=2)
par(cex.main = 3, cex.lab = 2, cex.axis = 2)
hist(x1, breaks=10, prob=TRUE, xlim=c(-4,6), col=rgb(1,0,0,0.3), main='2つの正規分布', xlab='値')
hist(x2, breaks=10, prob=TRUE, add=TRUE, col=rgb(0,0,1,0.3))
```

hist 関数で add = TRUE オプションを用いる

サンプルコード

サンプルコード1: 2つの正規分布の密度を重ね描き

```
set.seed(1)
```

```
x1 <- rnorm(30, mean=0)
```

```
x2 <- rnorm(30, mean=2)
```

→ 2種類の正規分布から2種類のデータセットを生成

```
par(cex.main = 3, cex.lab = 2, cex.axis = 2)
```

```
hist(x1, breaks=10, prob=TRUE, xlim=c(-4,6), col=rgb(1,0,0,0.3), main='2つの正規分布', xlab='値')
```

```
hist(x2, breaks=10, prob=TRUE, add=TRUE, col=rgb(0,0,1,0.3))
```

1つ目のヒストグラム

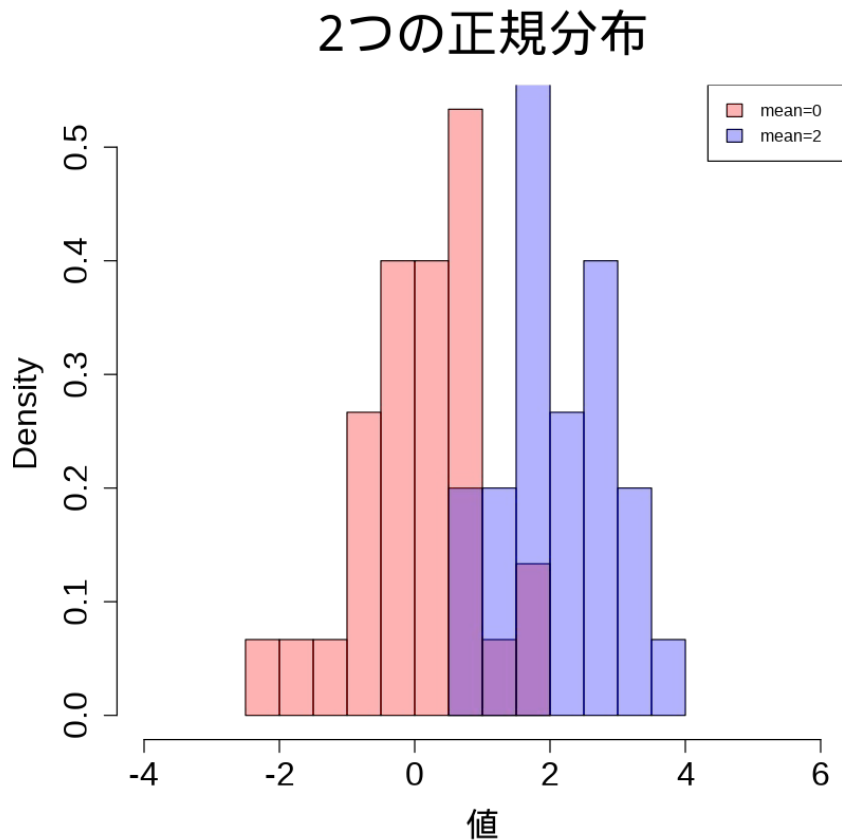
2つ目のヒストグラム

2つ目のヒストグラム中で add=TRUE
オプションを使って重ね描きする

hist 関数で add = TRUE オプションを用いる

実行結果

演習 : RStudioでコードを実行してプロットを重ねて表示してみよう



散布図

散布図: 基本 –二つの連続変数の関係の視覚化–

二つの連続変数の関係を点で示す最も基本的なグラフ

- 散布図の点は、対応する二つの連続変数の値の組み合わせを表す
- 二つの変数の間のパターンや関連性をおおまかに把握できる

Rでの基本操作: `plot()` 関数

```
plot(x="散布図のx軸の座標に指定したいベクトル名",  
     y="散布図のy軸の座標に指定したいベクトル名")
```

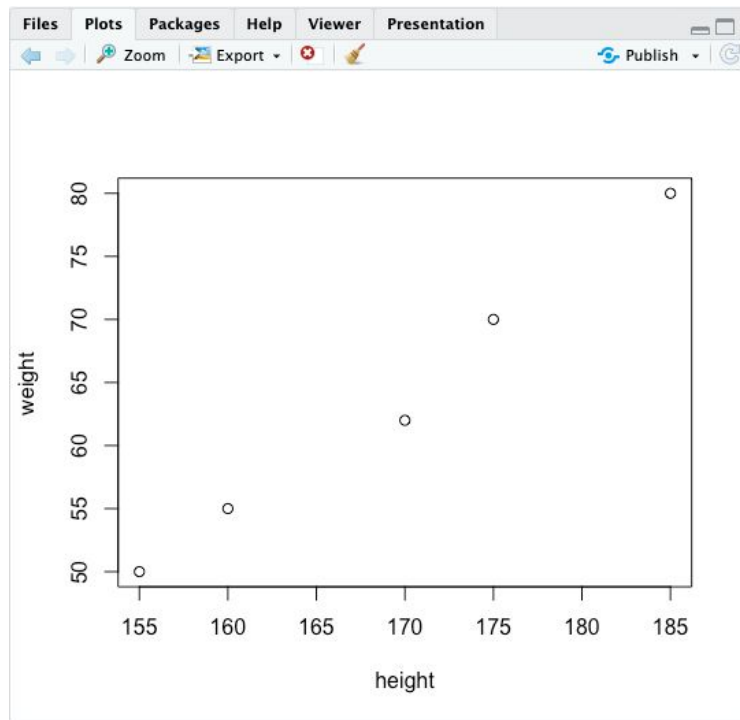
注意点

- x軸とy軸に指定するベクトルの長さは同じである必要がある
- 引数名 `x=` と `y=` は省略して書かれることも多い

散布図: 基本 -コード例-

あるグループの人々の身長と体重の散布図の作成を想定

```
# 身長をx軸、体重をy軸にした散布図
height <- c(155, 170, 175, 160, 185)
# 体重のデータ (単位: kg)
weight <- c(50, 62, 70, 55, 80)
# 身長をx軸、体重をy軸にした散布図
plot(x = height, y = weight)
```



散布図: タイトルとラベル

適切なタイトルと軸ラベル: 図の正確な解釈に不可欠

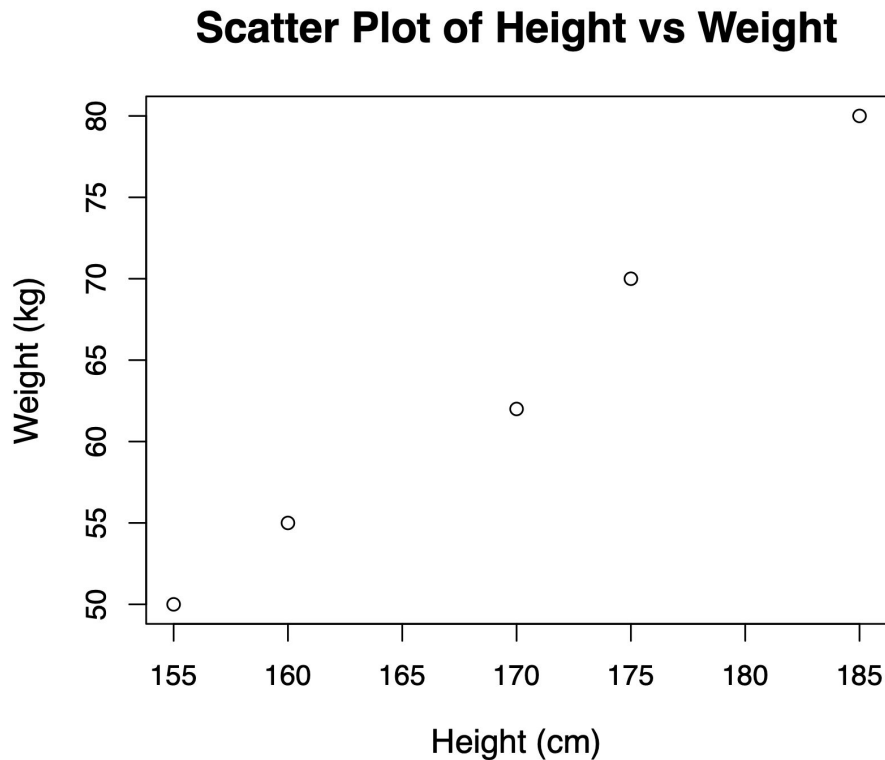
- タイトルとラベルは`plot()`関数の以下の引数を文字列で指定
 - `main`: 図のグラフタイトル
 - `xlab`: x軸ラベル
 - `ylab`: y軸ラベル
- 文字の大きさは`plot()`関数の以下の引数を数値で指定
 - `cex.main`: タイトルの大きさ
 - `cex.lab`: 軸ラベルの大きさ
 - デフォルトは1で指定する数値は拡大率

散布図: タイトルとラベル-コード例-

```
# 身長をx軸、体重をy軸にした散布図
plot(x = height, y = weight,
     main = "Scatter Plot of Height vs Weight", # タイトル
     xlab = "Height (cm)", # x軸ラベルの追加
     ylab = "Weight (kg)", # y軸ラベルの追加
     cex.main = 1.5, # タイトルは 1.5 倍のサイズ
     cex.lab = 1.2) # 軸ラベルは 1.2 倍のサイズ
```

演習: 上記のコードを実行してタイトルとラベルを追加したグラフを作成しよう

散布図: タイトルとラベルー出力例ー



散布図: 重ね書き

複数のグループの散布図に重ね書き: 視覚的なグループ間の比較

`plot()` 関数で散布図を描画した後

`points()` 関数により別の散布図を同じグラフに追加

デフォルトでは後から追加した点と最初のプロットの点の区別が難しい
グループの区別のため、以下の引数で色や形状を変更

- `col`: 点の色を変更(例: `col=2`, `col="red"`)
- `pch`: 形状の変更(例: `pch=17`, `col="*"`)

```
points(x="重ね書きのx軸の座標に指定したいベクトル名",  
       y="重ね書きのy軸の座標に指定したいベクトル名",  
       col = 数値 or "色名",  
       pch = 数値 or "文字")
```

色の指定

- `col=数値`による指定と色の対応

1	2	3	4	5	6	7	8
							

- 文字列による指定: `colors()`関数で一覧を取得できる
- 16進数カラーコードでも指定できる(`col="#DF536B"`)

点の形状の指定

- `pch=数値`による指定と形状の対応

0	1	2	3	4	5	6	7	8
								
9	10	11	12	13	14	15	16	17
								
18	19	20	21	22	23	24	25	
								

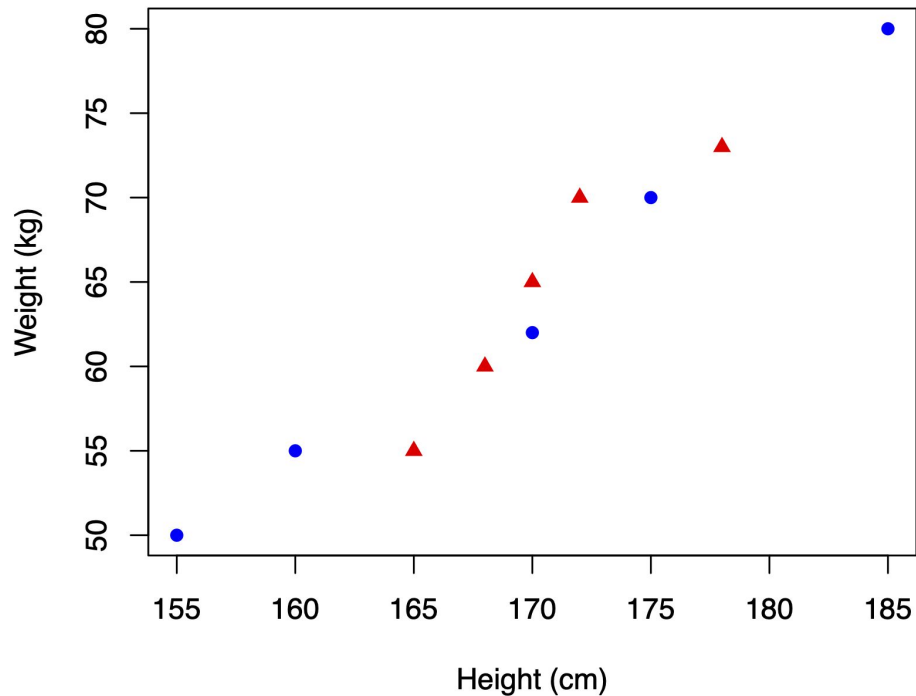
- 21-25 は塗りつぶしが可能な形状
- `pch` に文字を指定するとその文字がプロットされる(`pch="A"`)

散布図: 重ね書き -コード例-

```
# グループAの身長と体重
height_A <- c(155, 170, 175, 160, 185)
weight_A <- c(50, 62, 70, 55, 80)
# グループBの身長と体重
height_B <- c(168, 170, 178, 165, 172)
weight_B <- c(60, 65, 73, 55, 70)
# 最初の散布図 (グループA)
plot(x = height_A, y = weight_A,
     main = "Height vs Weight of Group A and B",
     xlab = "Height (cm)", ylab = "Weight (kg)",
     col = "blue", # グループAの点を青色に
     pch = 16) # グループAの点を塗りつぶし丸に
# グループBの点を重ね書き
points(x = height_B, y = weight_B,
      col = "red", # グループBの点を赤色に
      pch = 17) # グループBの点を塗りつぶし三角に
```

散布図：重ね書き -出力例-

Height vs Weight of Group A and B



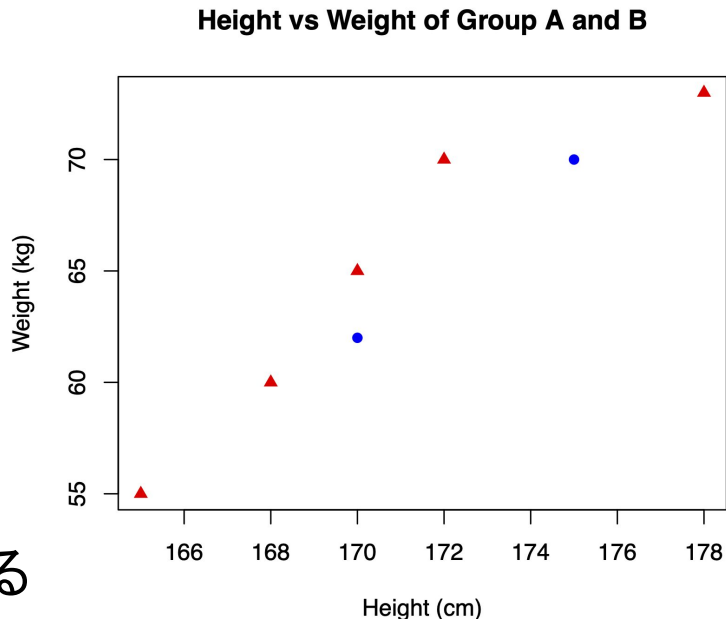
散布図: 描画範囲

重ね書き時の x軸範囲, y軸範囲 の指定について

- `points()` 関数は点の描画のみ
- `plot()` 関数でグラフを描画した場合
後から `points()` 関数で描画しても
描画範囲は変わらない

描画範囲外に点が存在する例:

- グループBを先に`plot()` 関数で
描画した場合
グループAの $x = 155, 185$ の2点が消える



散布図: 描画範囲 – 軸範囲の指定 –

重ね書きする全ての点が適切に表示されるように
最初に `plot()` 関数で適切な範囲を設定することが必要

- `xlim`: x軸範囲を最小値と最大値のベクトルで指定(`xlim=c(40,80)`)
- `ylim`: y軸範囲を最小値と最大値のベクトルで指定(`ylim=c(140,180)`)

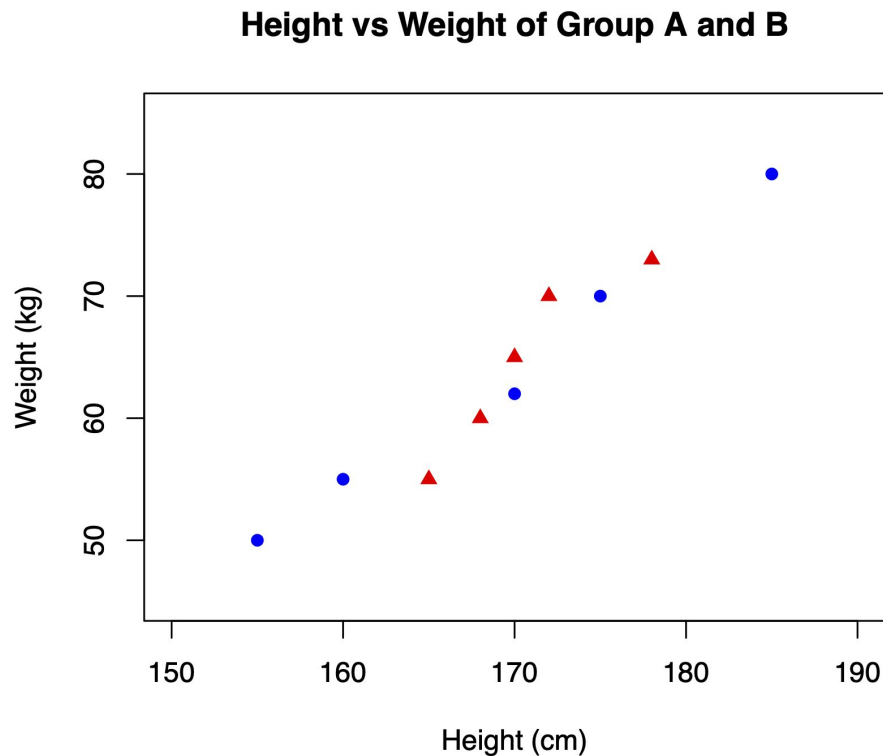
```
plot(..., xlim = x軸の最小値と最大値の長さ2のベクトル,  
      ylim = y軸の最小値と最大値の長さ2のベクトル)
```

散布図: 軸範囲の指定 -コード例-

演習: 上記のコードを実行してグラフに軸範囲を設定してみよう

```
# 軸範囲の設定
plot(x = height_B, y = weight_B,
     main = "Height vs Weight of Group A and B",
     xlab = "Height (cm)", ylab = "Weight (kg)",
     col = "red", pch = 17,
     xlim = c(150,190), ylim = c(45,85)) # 軸範囲の設定
points(x = height_A, y = weight_A, col = "blue",
       pch = 16)
```

散布図：軸範囲の指定 -出力例-



散布図: 凡例の追加

- 複数グループを可視化した散布図に
凡例を付けるには `legend()` 関数を使う

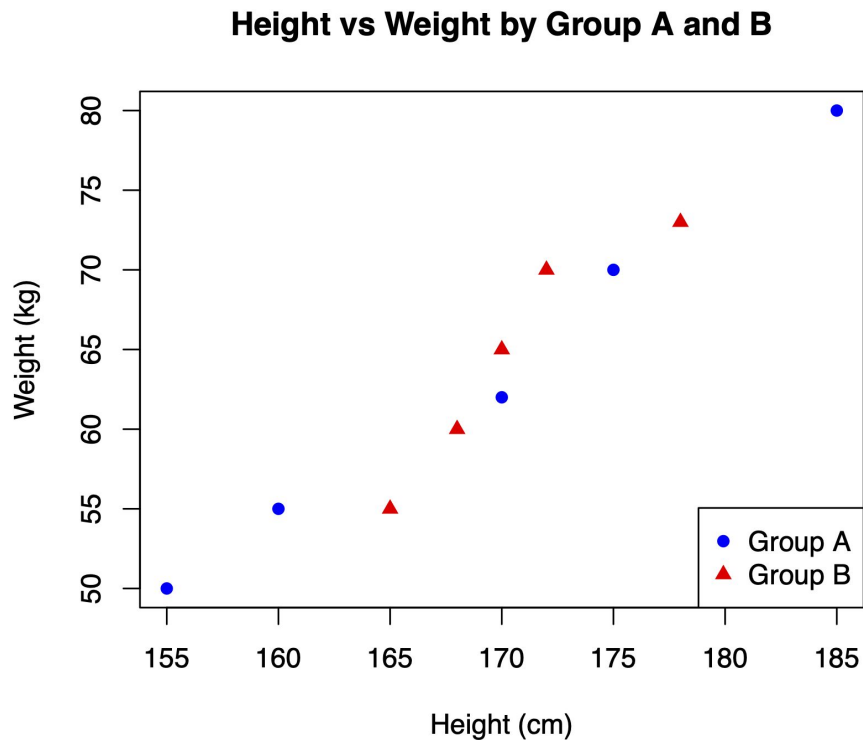
主な引数

- `legend`: 表示するラベルの文字列ベクトル
- `col`: ラベルに対応する点や線の色
- `pch`: 点の記号
- `lty`: 線の種類(線も描画する場合)
- 位置指定: `x=`, `y=` または キーワード ("`topright`", "`bottomleft`" など)

散布図: 凡例の追加 -コード例-

```
# 散布図の作成
plot(x = height_A, y = weight_A,
     col = "blue", pch = 16,
     main = "Height vs Weight by Group A and B",
     xlab = "Height (cm)", ylab = "Weight (kg)")
points(x = height_B, y = weight_B,
       col = "red", pch = 17)
# 凡例の追加
legend("bottomright", # 右下に追加
      legend = c("Group A", "Group B"), # ラベルの指定
      col = c("blue", "red"), # 点の色の指定
      pch = c(16, 17)) # 点の形状の指定
```


散布図：凡例の追加－出力例－



(復習)図の保存

- 関数による実行

```
# 出力ファイルの設定
cairo_pdf("hist.pdf",width=7,height=7)
# 画像をPlot windowに出力するコード（複数行でも可）
print(g)
# ここまでの出力を保存
dev.off()
```

- R GUIによる実行

- 画面右下の「Plots」パネル上部の Export ボタンをクリック
- メニューから Save as Image または Save as PDF を選択
- フォーマット・幅・高さを指定
- 保存先・ファイル名を指定
- Save をクリックすると保存

演習問題1 –使用データ–

anorexia_df(MedDataSets パッケージ)

- 拒食症患者を対象としたランダム化比較試験(RCT)のデータ
- 被験者数:72名
- 3群(標準ケア、認知行動療法、家族療法)の割り付け
- 各患者の介入前後の体重変化を記録
- 変数
 - Prewt: 介入前体重(kg)
 - Postwt: 介入後体重(kg)
 - Treat: グループ
Cont: 対照群(標準ケア) / CBT: 認知行動療法群 / FT: 家族療法群

演習問題1 –図の作成–

介入前体重と介入後体重の関係を表す散布図を3つの群で点の色、形状を変えて作成してください。

1. MedDataSets からデータセット anorexia_df の読み込み
2. 各群、介入前後それぞれのベクトルの作成
3. 3群を重ね書きすることを考慮して描画範囲を決定
4. plot() 関数、points() 関数で3群の介入前体重と介入後体重の散布図を作成
5. legend() で凡例を追加

Rコード例 -過食症データの散布図作成-

```
# 1. データの読み込み
install.packages("MedDataSets") # 初回のみ
library(MedDataSets)
data("anorexia_df")
df <- anorexia_df

# 2. 各群のベクトルを作成
Prewt_Cont <- df$Prewt[df$Treat == "Cont"]
Postwt_Cont <- df$Postwt[df$Treat == "Cont"]
Prewt_CBT <- df$Prewt[df$Treat == "CBT"]
Postwt_CBR <- df$Postwt[df$Treat == "CBT"]
Prewt_FT <- df$Prewt[df$Treat == "FT"]
Postwt_FT <- df$Postwt[df$Treat == "FT"]

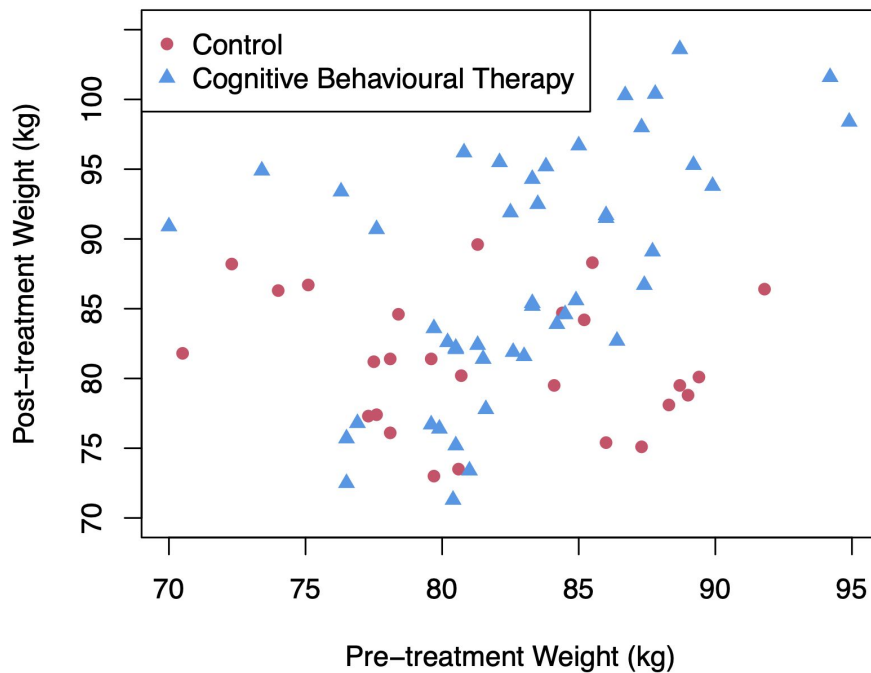
# 3. 描画範囲の算出
x_range = range(Prewt_Cont, Prewt_CBT, Prewt_FT)
y_range = range(Postwt_Cont, Postwt_CBR, Postwt_FT)
```

Rコード例 - 過食症データの散布図作成 -

```
# 4. plot() 関数、points() 関数で3群の介入前体重と介入後体重の散布図を作成
# plot() 関数によるCont群のプロットとタイトル等の設定
plot(Prewt_Cont, Postwt_Cont,
     col = 2,      # 赤
     pch = 16,     # 塗りつぶし丸
     main = "Anorexia: Post vs Pre Weight by Group",
     xlab = "Pre-treatment Weight (kg)",
     ylab = "Post-treatment Weight (kg)",
     xlim = x_range, ylim = y_range)
# points() 関数による重ね書き
points(Prewt_CBT, Postwt_CBR,
       col = 3,     # 緑
       pch = 17)    # 塗りつぶし四角
points(Prewt_FT, Postwt_FT,
       col = 4,     # 青
       pch = 18)    # 塗りつぶし三角
# 5. 凡例の追加
legend("topleft",
      legend = c("Control", "Cognitive Behavioural Therapy", "Family Therapy"),
      col = c(2, 3, 4), pch = c(16, 17, 18))
```

出力例 -過食症データの散布図作成-

Anorexia: Post vs Pre Weight by Group



時系列プロット

時系列プロット: 時間的変化の視覚化

患者のバイタルデータ(心拍数、血圧など)、薬の血中濃度、感染症の患者数など、多くのデータが時系列データとして扱われる

同一のサンプルの症例を線でつなぐことがよく行われる

Rでの基本操作: `plot()` においてグラフの種類を指定 (`type = "l"`)

```
plot(x = "x軸(時間軸)の座標に指定したいベクトル名",  
     y = "y軸の座標に指定したいベクトル名",  
     type = "l") # type="l": 折れ線グラフ (line)
```

- x軸は数値以外に、時間を表す形式(Date型、POSIXct/lt型など)でも良い
- y軸のデータは数値ベクトル

plot()関数の引数typeについて

- "p": 点 (points)
- "l": 線 (lines)
- "b": 点と線 (both)
- "c": 点を描かず線だけを描く (connected lines)
「"b" から "p" を除いたもの」
- "o": 点と線を重ねて表示 (overplotted)
- "h": 各点からx軸への垂直線 (histogram-like vertical lines)
- "s": 階段状のグラフ (steps), y軸の値がx軸の値から次のx軸の値まで同じ
- "S": もう一つの階段状のグラフ, y軸の値がx軸の値から前のx軸の値まで同じ
- "n": 何も描画しない (no plotting)
後から要素を追加する場合に便利

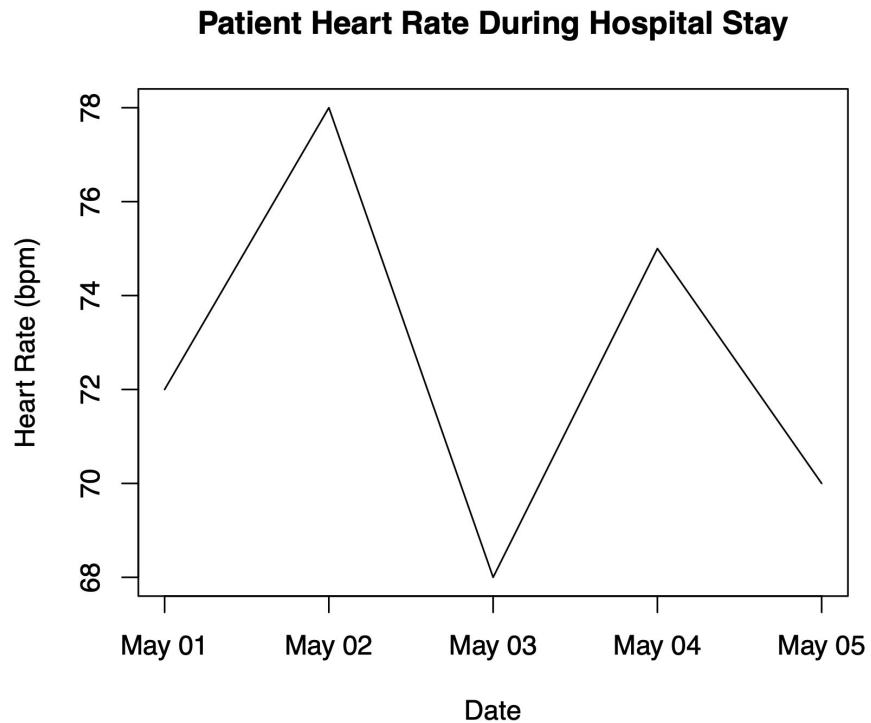
時系列プロット: 線の種類と点の追加

散布図と同様に、タイトル、ラベルを設定できる

```
# 日付データ
dates <- as.Date(c("2025-05-01", "2025-05-02", "2025-05-03",
"2025-05-04", "2025-05-05"))
# 心拍数データ (単位: 回/分)
heart_rate <- c(72, 78, 68, 75, 70)

# 日付をx軸、心拍数をy軸にした時系列プロット
plot(x = dates, y = heart_rate, type = "l",
     main = "Patient Heart Rate During Hospital Stay",
     xlab = "Date",
     ylab = "Heart Rate (bpm)")
```

時系列プロット: 線の種類と点の追加



時系列プロット: 重ね書き

散布図と同様に `plot()` 関数の後に `points()` 関数により重ね書きができる

- デフォルトでは後から追加した点と最初のプロットの点の区別が難しい

グループの区別のため、以下の引数で色や線の種類を変更

- `col`: 点の色を変更(例: `col=2`, `col="red"`)
- `lty`: 線の種類の変更(例: `lty=2`, `lty="dotted"`)

```
points(x="重ね書きのx軸の座標に指定したいベクトル名",  
       y="重ね書きのy軸の座標に指定したいベクトル名",  
       col = 数値 or "色名",  
       lty = 数値 or "文字")
```

線の種類の指定

lty=の指定と線の種類の対応

0	blank	
1	solid	
2	dashed	
3	dotted	
4	dotdash	
5	longdash	
6	twodash	

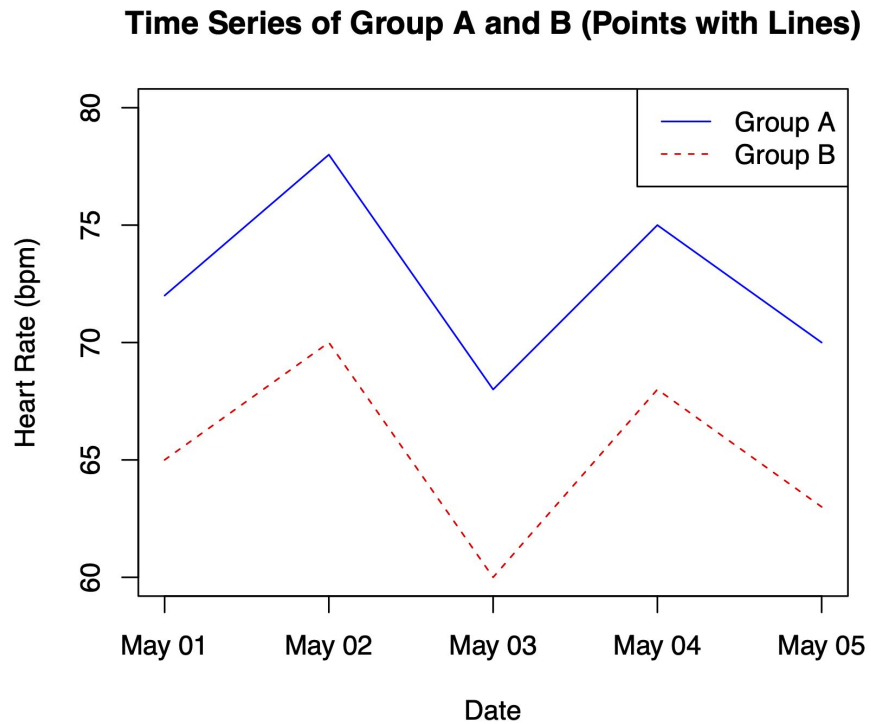
時系列プロット: 例

```
# グループAの日付と値
dates_A <- as.Date(c("2025-05-01", "2025-05-02", "2025-05-03",
"2025-05-04", "2025-05-05"))
values_A <- c(72, 78, 68, 75, 70)
# グループBの日付と値
dates_B <- as.Date(c("2025-05-01", "2025-05-02", "2025-05-03",
"2025-05-04", "2025-05-05"))
values_B <- c(65, 70, 60, 68, 63)
# 最初の時系列プロット（グループA、線で表示）
plot(x = dates_A, y = values_A, type = "l",
     main = "Time Series of Group A and B (Points with Lines)",
     xlab = "Date", ylab = "Heart Rate (bpm)",
     col = "blue", lty = 1, # グループAの線を青色の実線に
     ylim = c(60,80)) # y軸範囲の設定
```

時系列プロット: 例 続き

```
# グループBの時系列データを点を線で重ね書き
points(x = dates_B, y = values_B, type = "l",
       col = "red", lty = 2) # グループBの線を赤色の破線に
# 凡例の追加
legend("topright", legend = c("Group A", "Group B"),
      col = c("blue", "red"), lty = c(1, 2))
```


時系列プロット: 例 出力



演習問題 –使用データ–

- 使用データ: epil (MASS パッケージ)
- データ概要:
 - 59人のてんかん患者を対象とした臨床試験データ
 - 各被験者について、介入後2週間ごとの発作回数(計4時点)を記録
 - 介入はプログアバイド(progabide)とプラセボ
- 変数
 - subject: 被験者ID
 - period: 観察時期(1~4)
 - seizure: 発作回数
 - treatment: 治療群(placebo / progabide)

今回の演習では「被験者1」と「被験者2」のデータを比較

演習問題 –グラフ作成–

以下の手順に従って、時系列プロットを作成してください。

1. epil データから被験者1と2のデータを取り出す
2. plot() 関数で被験者1のデータを青の実線で描画
3. points() 関数で被験者2のデータを赤の破線で重ねて描画
4. legend() で凡例を追加

Rコード例 -データ抽出-

```
# データの読み込み
install.packages("MASS") # 初回のみ
library(MASS)
data("epil")
```

```
# 被験者1と2のデータを抽出
df1 <- epil[epil$subject==1,]
df2 <- epil[epil$subject==2,]
```

- データ抽出はsubset("データセット名","条件")でも可能
 - 条件は論理式(例: subject==1)で指定
 - 論理式で変数名はそのまま使用できる(\$マーク不要)

```
df1 <- subset(epil,subject==1)
```

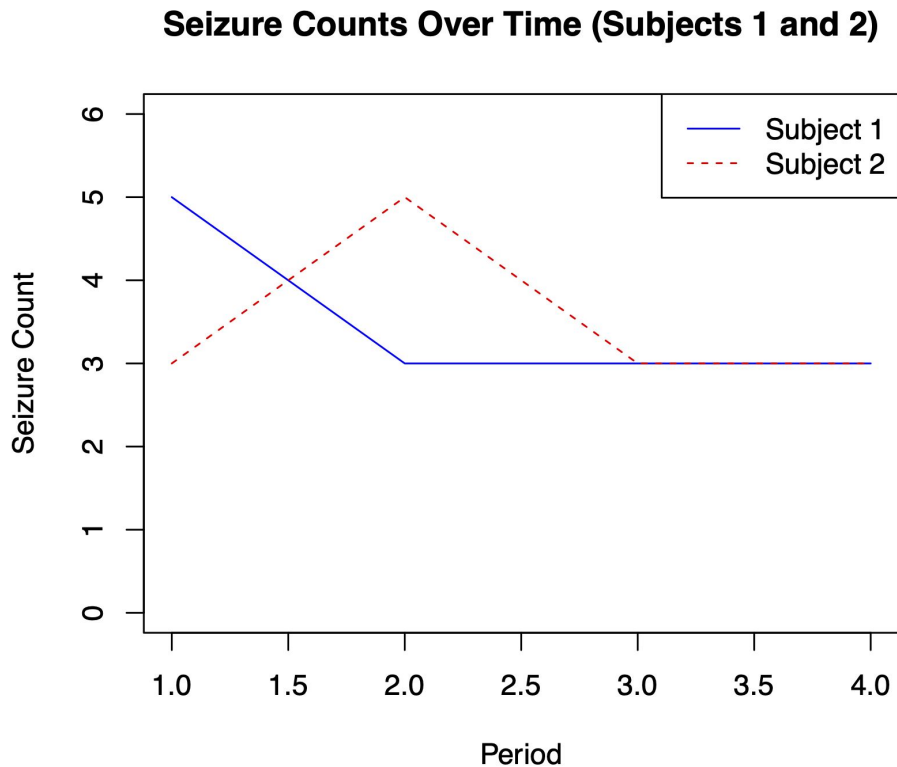
Rコード例 –被験者1と2の発作回数のプロット–

```
# 被験者1をプロット（青の実線）
plot(df1$period, df1$y, type = "l",
     col = "blue", lty = 1,
     ylim = c(0, 6),
     xlab = "Period", ylab = "Seizure Count",
     main = "Seizure Counts Over Time (Subjects 1 and 2)")
```

```
# 被験者2を重ね書き（赤の破線）
points(df2$period, df2$y, col = "red", type="l", lty = 2)
```

```
# 凡例の追加
legend("topright",
     legend = c("Subject 1", "Subject 2"),
     col = c("blue", "red"), lty = c(1, 2))
```

出力例 - 発作回数の時系列プロット -



for文による繰り返し処理の基本

for文は、繰り返し処理(loop)を行うための構文

- ・同じ処理を複数のデータに対して繰り返したいときに使用

基本構文:

```
for (i in 1:5) {  
    # i を使った処理をここに書く  
}
```

- ・この例では、i が 1 から 5 まで順に値を取り
- ・中括弧 {} 内の処理が5回繰り返される

for文の簡単な例 – 1～5の二乗を表示–

次のコードは、1～5 の数を順に取り出し、それぞれの 2乗 を表示

```
for (i in 1:5) {  
    print(i^2)  # iの2乗を表示  
}
```

出力結果：

```
[1] 1  
[1] 4  
[1] 9  
[1] 16  
[1] 25
```

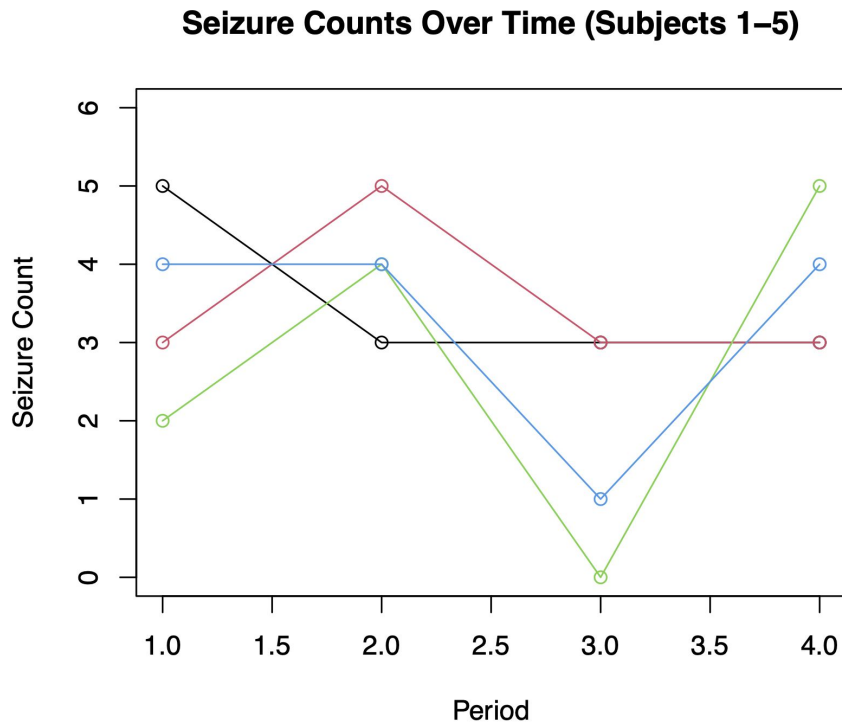

コード例：被験者1～5の発作回数の重ね書き

- ・最初に plot() で軸・範囲・ラベルだけを描画 (type = "n")
- ・for文を使って、被験者1～5を points() で1人ずつ重ね書き

```
# 空のプロットを用意 (type = "n")
plot(NA, type = "n",
     xlim = c(1, 4), ylim = c(0, 6),
     xlab = "Period", ylab = "Seizure Count",
     main = "Seizure Counts Over Time (Subjects 1-5)")

# 被験者1～5のデータを順に描画
for (i in 1:5) {
  dfi <- subset(epil, subject == i)
  points(dfi$period, dfi$y, type = "o", col = i)
}
```

出力例 –被験者1～5の発作回数の重ね書き–



演習問題 –繰り返し処理や層別による重ね書き–

データセット epil の視覚化

1. epil データセットにおいて placebo 群と progabide 群で色と線の種類を変えて全ての被験者の発作回数のperiod 1–4 の推移を重ね書きしてください
2. epil データセットにおいて placebo 群と progabide 群で色と線の種類を変えて各群の被験者の発作回数のピリオドごとの平均の推移を重ね書きしてください

Base Rを用いたプロット 詳細な設定

(参考) ts クラスによる時系列データの管理

ts クラス: 時系列データ専用のデータ構造

- ・主に等間隔の時系列データ(例: 月次、四半期、年次)に適用
- ・開始時点や頻度などの時系列情報を内部に保持
- ・グラフ描画や時系列解析の関数で自動的に利用される

(参考) ts() 関数の構文と使用例

ts() 関数: ベクトルや行列に時系列情報を付加し、ts クラスのオブジェクトを作成

- data: 数値ベクトルまたは行列
- start: 開始時点 (例: 年次: 1980、月次: c(2023, 1))
- frequency: 周期 (年次=1, 四半期=4, 月次=12 など)

```
ts(data, start = 開始時点, frequency = 頻度)
# 月次売上データ (2023年1月~2024年12月まで)
sales <- c(100, 120, 130, 125, 140, 135, 150, 155, 160,
          158, 165, 170, 175, 180, 178, 185, 190, 195,
          200, 198, 205, 210, 215, 220)
# tsオブジェクトとして作成 (開始: 2023年1月、頻度: 月次)
ts_sales <- ts(sales, start=c(2023, 1), frequency=12)
# プロット (横軸に年月の情報が自動表示される)
plot(ts_sales,
     main = "Monthly Sales (2023-2024)", ylab = "Sales")
```

(参考) ts クラスで使える主な関数

ts クラスは多くの専用関数に対応しており
時系列解析の基礎的な操作が簡単に実行可能

- plot(): 時系列データのグラフ描画 (自動で時間軸が付く)
- window(): 特定の期間を取り出す (例: 2010年以降)
- cycle(): 季節要素の確認 (周期的データに有用)
- frequency(): データの周期 (frequency) の取得
- start(), end(): 開始点・終了点の取得
- lag(), diff(): ラグ (遅れ) や差分系列の生成
- aggregate(): 期間単位での集計 (例: 月次 → 年次)

より高度な解析は forecast や fable パッケージを使用

(参考) plot() 関数の他の引数

1. 軸の調整

- log: 対数軸の指定 ("x", "y", "xy")

2. 色とサイズ

- bg: プロット記号の背景色 (pch = 21 から 25 でのみ反映)
- cex: 点のサイズの倍率 (デフォルト=1)

3. タイトル、軸ラベル

- sub: グラフサブタイトル
- cex.main, cex.lab, col.lab, font.sub など: 文字のサイズや色

(参考) plot() 関数の他の引数

4. 軸や枠の制御

- axes: 軸の描画 (FALSEで非表示)
- ann: ラベル・タイトルの描画 (FALSEで非表示)
- frame.plot: 枠線の表示 (TRUE/FALSE)
- xaxt, yaxt: 特定の軸のみ抑制 ("n"で非表示)

5. その他の便利な設定

- panel.first, panel.last: プロット前後の実行命令
- asp: 縦横比 (例: asp = 1で等間隔)
- xgap.axis, ygap.axis: 目盛りとプロット領域の隙間設定

(参考) par() 関数とマルチパネル表示

- `mfrow = c(nrow, ncol)`: 描画順が行優先のパネル分割
- `mfcol = c(nrow, ncol)`: 描画順が列優先のパネル分割
- `mar = c(bottom, left, top, right)`: プロット内余白
- `oma = c(bottom, left, top, right)`: 全体外余白

```
# 2列横並びに分割
par(mfrow = c(1,2), mar = c(4,4,2,1))
# 左パネル: 対照群
plot(Prewt_Cont, Postwt_Cont, main = "Control",
     xlab = "Prewt (kg)", ylab = "Postwt (kg)",
     col = 2, pch = 16)
# 右パネル: CBT 群
plot(Prewt_Treat, Postwt_Treat, main = "CBT",
     xlab = "Prewt (kg)", ylab = "Postwt (kg)",
     col = 4, pch = 17)
# リセット
par(mfrow = c(1,1))
```

(参考) par() 関数の他の引数の紹介

par() 関数では、描画の詳細を制御する多くのオプションが指定可能(以下一部)

- las: 軸ラベルの方向 (0: 水平, 1: 垂直, 2: 全て垂直, 3: 全て水平)
- tck: 目盛り線の長さ(負の値で内向き)
- cex, cex.axis, cex.lab, cex.main: 文字サイズの拡大率
- col.axis, col.lab, col.main: 軸・ラベル・タイトルの色
- xaxt, yaxt: x軸/y軸を描画しない ("n" 指定で非表示)

設定できるパラメータと現在の設定はpar()で表示できる

```
par()
```

par() の設定は R セッション全体に影響するため、必要に応じてリセットするのが推奨される

(参考) plot() の 引数のベクトル指定

- col、pchなどはグループに対応するベクトルで指定できる
- 辞書となるベクトルを作成しておくにより簡単に指定できる
 - Treat が factor の場合、levels(df\$Treat) の順序に注意
 - 必要に応じて as.character() や relevel() を用いて明示的に制御する
 - ifelseなどを用いてそれぞれ色を表すベクトルにしてもよい
- long formatのデータで特に有用

```
# 辞書となるベクトルの定義
cols <- c(Cont = "red",   CBT = "blue")
pchs <- c(Cont = 16,      CBT = 17)
plot(df$Prewt, df$Postwt,
     col = cols[df$Treat],
     pch = pchs[df$Treat],
     xlab = "Pre-treatment Weight",
     ylab = "Post-treatment Weight")
```

(参考) ベクトル指定を用いた散布図例

```
plot(df$Prewt, df$Postwt,  
     col = cols[df$Treat],  
     pch = pchs[df$Treat],  
     main = "Anorexia: Post vs Pre Weight",  
     xlab = "Pre-treatment Weight",  
     ylab = "Post-treatment Weight")  
legend("topleft",  
      legend = names(cols),  
      col     = cols,  
      pch     = pchs)
```

(参考) 色に透明度を指定する

多くの点が重なり合う散布図で視認性向上に使える

- `rgb(r, g, b, alpha)` 例:`rgb(1,0,0,0.4)`
- `adjustcolor("blue", alpha.f=0.3)`

```
plot(df$Prewt, df$Postwt,  
      col = rgb(1,0,0,0.4), pch = 16,  
      xlab = "Pre-treatment Weight",  
      ylab = "Post-treatment Weight")  
# または  
plot(df$Prewt, df$Postwt,  
      col = adjustcolor("blue", alpha.f = 0.5),  
      pch = 16)
```

(参考) abline() による直線の上書き

- `abline(h = y0)`: 水平線
- `abline(v = x0)`: 垂直線
- `abline(a, b)`: 直線 $y = a + b x$
- `abline(lm(y ~ x))`: 回帰直線

```
plot(df$Prewt, df$Postwt, pch = 16, col = "gray")
# 45度線を追加
abline(a = 0, b = 1, lty = 2, col = "black")
# 回帰直線を追加
fit <- lm(Postwt ~ Prewt, data=df[df$Treat=="Cont", ])
abline(fit, col = "red", lwd = 2)
```

(参考) lines(), curve() の紹介

- lines() 関数:
 - points(..., type = "l") とほぼ同じ用途だが、lines() は線専用
 - 点の描画はできない(pch は使えない)
 - 実線・破線や太さ(lty, lwd)の指定が可能
- curve() 関数:
 - 数式や関数を直接指定して関数曲線を描画できる
 - 実測値ではなく、理論式やモデル式の描画に便利
 - curve() は新規描画と重ね書き(add = TRUE)の両方に対応

```
lines(x, y, col = "blue", lty = 2)
curve(log(x + 1), from = 0, to = 10, col = "red")
```


(参考) axis() による軸の設定

- axis(side, at, labels, las, tck)
 - side: 1=下, 2=左, 3=上, 4=右
 - at: 目盛の位置(数値ベクトル)
 - labels: 目盛ラベル(文字ベクトル)
 - las: ラベルの向き(0-3)
 - tck: 目盛線の長さ(絶対値または割合)

```
# 軸なしでプロット
plot(df$Prewt, df$Postwt, axes = FALSE, ann = FALSE, pch = 16)
# 下軸
axis(1, at = seq(70, 95, by = 5),
     labels = seq(70, 95, by = 5), las = 1,
     tck = -0.02)
# 左軸
axis(2, at = seq(70, 105, by = 5), las = 1)
box() # 枠線を追加
```

(参考) 連続変数とカテゴリカル変数の2変数の図示

- ・boxplot() 関数は、連続変数とカテゴリカル変数の関係を視覚的に示すのに便利

```
boxplot(連続変数 ~ カテゴリカル変数, data=データ名)
```

- ・例:治療群(trt)ごとの発作回数(y)の分布を比較

```
boxplot(y ~ trt,  
        data = epil,  
        xlab = "Treatment group",  
        ylab = "Number of seizures")
```